

Expanding Logical Space Freely: A Memory-efficient Mapping Table Design for Compressional SSDs

Zixuan Huang[†], Tianyu Wang^{*‡}, Kecheng Huang[†], Zelin Du[†], Zili Shao[†]

[†]The Chinese University of Hong Kong, Hong Kong, China

[‡]Shenzhen University, Shenzhen, China

{zxhuang, kchuang21, zldu22, shao}@cse.cuhk.edu.hk, tywang@szu.edu.cn

Abstract—Compressional SSDs can be a mixed blessing. While they offer users expanded logical space beyond the physical capacity, they complicate the Flash Translation Layer (FTL) design by requiring a larger Logical-to-Physical (L2P) address mapping, which places a heavier burden on the limited in-SSD memory, leading to a degraded I/O performance. In this paper, we aim to reduce the memory footprint of the L2P mapping table in compressional SSDs by proposing a novel N-to-1 L2P mapping table design that consolidates multiple logical entries into a single entry. This approach eliminates the duplication of physical page numbers when a physical page contains several compressed logical pages. To accommodate the dynamic compression ratios of real-world workloads, we introduce promotion and demotion that enable mapping table entries to migrate between pages with different compression ratios. Additionally, to address the issue of partial invalidation—where some compressed logical pages within a physical page are invalid due to the N-to-1 mapping—we present a compression-aware garbage collection algorithm aimed at minimizing the number of copy operations for partially invalid physical pages. We have implemented our design in MQsim, a widely used SSD simulator, and have conducted a series of experiments to evaluate the effectiveness of the proposed techniques. The results demonstrate that our approach significantly reduces the mapping table size in compressional SSDs, leading to an improved mapping table cache hit ratio and a reduced I/O latency compared to traditional compressional SSDs.

Index Terms—Compressional SSD, FTL, Cache management

I. INTRODUCTION

The rapid growth of data volume has significantly increased the demand for high-capacity SSDs [21], [35]. However, due to the diminishing marginal effect, with Multi-Level-Cell (MLC)-based SSDs advancing to 5-bit or even 6-bit levels, the pace of capacity development begins to slow down [3], [7]. As an alternative approach, compressional SSDs emerge to offer users more available storage space than the actual physical space by compressing multiple logical pages into one physical page [2], [13]. Although compressional SSDs excel in workloads with higher compression ratios (CRs), they disrupt the traditional one-to-one Logical-to-Physical (L2P) address mapping by allowing multiple logical pages to be mapped to the same physical page. This complicates SSD firmware design.

With compressional SSDs, a larger logical space directly increases the size of the L2P mapping table, placing even more strain on limited in-SSD memory. For instance, if an 8TB compressional SSD with a 4KB page size exposes 16TB of logical space to users, the entire L2P mapping table will be doubled

to 16GB which is a significant overhead for the limited in-SSD memory. Thus, existing approaches usually store the mapping table in NAND flash and cache it in the in-SSD memory. Considering that the user data is also cached in the memory, a larger mapping table inevitably compromises the cache hit ratio, thus affecting the I/O performance of compressional SSDs as more I/O operations require reading the mapping page from NAND flash before accessing corresponding data pages.

We have observed that inside compressional SSDs, a low cache hit ratio significantly prolongs access time, while the compression logic has a comparatively minor impact on overall latency [39]. This diminishes the effectiveness of existing studies that focus on optimizing compression algorithms [4], [6], [19], [22], [28], [37] or workflows [15], [17], [18], [20], [24], [30], [32], [36]. Therefore, in this paper, we aim to design a memory-efficient N-to-1 L2P mapping table for compressional SSDs. Specifically, our key idea is to eliminate redundancy in the traditional one-to-one L2P mapping table, where a Physical Page Number (PPN) can appear in multiple entries as in compressional SSDs multiple logical pages can be mapped to the same physical page. In our N-to-1 mapping table, consecutive N logical pages are consolidated into a single entry, efficiently eliminating the redundancy of physical page information. To achieve this, several challenges need to be conquered.

First, since different types of data exhibit varying CRs, a fixed parameter N in the N-to-1 mapping table cannot accommodate all scenarios effectively. Therefore, supporting diverse parameter N settings in the mapping table becomes the primary challenge. To address this, we introduce an additional marker in each Global Translation Directory (GTD) entry. The entry corresponds to the parameter N. It indicates how many consecutive logical pages can be mapped through a single second-level entry in the corresponding translation page based on the compression ratio of data. This allows us to efficiently support varying compression ratios for different workloads.

Second, as each entry represents multiple logical pages, traditional translation page allocation scheme becomes ineffective. With one translation page containing multiple entries with various CRs, logical space fragmentation becomes another critical challenge. To efficiently manage entries in a translation page, we introduce the concept of Translation Chunk inside each translation page, enabling fine-grained management of translation page space. For instance, a translation page can have eight 8-to-1 chunks, four 4-to-1 chunks, two 2-to-1 chunks, one 1-to-1 chunk, or other combinations. Building upon this, we

*Corresponding author

propose a partial translation page allocation scheme with chunk granularity, which eliminates the need to adjust the logical space range within one directory entry. This makes the mapping table more flexible and efficient.

Third, since the data stored on each logical page may change, its compression ratio may also change correspondingly, presenting another challenge for our N-to-1 mapping table design. If data that initially has a higher CR is updated to a lower CR, the original N-to-1 mapping becomes invalid as more physical pages are needed to store the original data from the N logical pages. Thus, we propose promotion and demotion for the translation page to dynamically adjust the compression ratios of the entries within the translation page. When some data is updated with a lower CR (i.e., requiring more physical pages), we reduce the CR of the corresponding translation page to allow each entry to manage fewer logical pages. In contrast, if all the entries in a translation page are updated with higher CR data, we promote the page to a higher CR to maximize space utilization in the mapping table. Thus, we can effectively keep the mapping table size relatively small, achieving better memory efficiency.

The final challenge concerns garbage collection (GC). In compressional SSDs, since each physical page contains multiple compressed logical pages, the invalidation of any single logical page results in a partial invalidation of the entire physical page, making traditional GC algorithms less effective. Thus, we develop a compression-aware GC mechanism that delays the reallocation step and retains valid pages in memory, enabling potential recompression of new updates and leading to space savings. We also take this opportunity to assess whether the corresponding translation page can be promoted at that time.

We have implemented our design in MQsim [33], a widely used SSD simulator, and evaluate the effectiveness of the proposed techniques. Results show that we can reduce the mapping table size by 50%, and achieve 2.06x speedup in IO latency compared with DFTL [11].

Our contributions are summarized as follows:

- We design a memory-efficient N-to-1 mapping table tailored for compressional SSDs, which substantially reduces the memory footprint and enhances I/O efficiency.
- A partial translation page allocation scheme is introduced to address the logical space fragmentation issue.
- We achieve dynamic workloads support by presenting promotion and demotion for translation pages, cooperating with a compression-aware GC policy for efficient physical space usage.
- The proposed technique is implemented in MQsim and evaluated using diverse real-world workloads, demonstrating its effectiveness.

II. BACKGROUND AND MOTIVATION

A. Compressional SSDs

Compressional SSDs are solid-state drives incorporating compression techniques, often featuring hardware compression/decompression modules integrated directly into the SSDs [43]. This technology has gained significant attention in recent

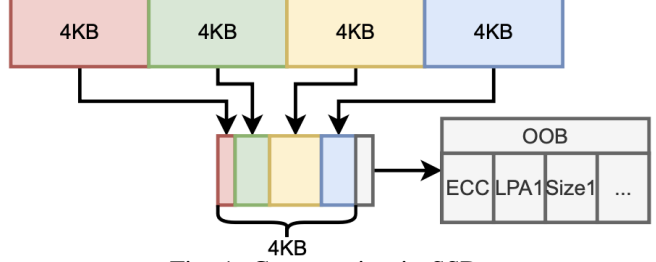


Fig. 1: Compression in SSDs.

years [12], [14], [31]. In this work, we focus on one type of compressional SSDs that support built-in transparent data compression capability, in which data are compressed at a fixed granularity. For example, compressional SSDs from ScaleFlux use a per-4KB compression scheme in which every 4KB logical page is compressed individually and then multiple compressed pages are stored together in one physical page as much as possible. Figure 1 illustrates this process, where four logical pages are compressed and stored together in a single physical page. Compared with host-side compression [1], [5], [26], [38], such in-storage transparent compression completely relieves host CPU from compressing and decompressing data. To facilitate decompression and reverse mapping, the corresponding metadata can be stored in the out-of-band (OOB) area of a physical page. This metadata contains information necessary for identifying the logical page address and its size during the decompression process. Moreover, the system maintains bitmaps for every physical page, indicating valid logical pages for garbage collection.

B. Mapping Table Issues

Inside the FTL, an L2P mapping table is maintained for address translation. Constrained by limited in-SSD memory space, DFTL proposes on-demand page-level mapping schemes. The mapping table is stored in flash memory and cached in DRAM. A Global Translation Directory (GTD) is needed for indexing, which tracks the mapping of the translation pages. DFTL significantly reduces the amount of memory needed. This idea works well in traditional SSDs [16], [23], [27], [29], [42]. However, when applied to compressional SSDs, it does not address the problem of physical page number redundancy. In compressional SSDs, multiple logical pages are stored within a single physical page, resulting in consecutive entries in the mapping table that store the same physical page number. This leads to a waste of space and low cache hit ratios, which severely degrade the performance of the FTL.

C. Garbage Collection Issues

With the out-of-place update characteristic of flash memory, old physical pages become invalid once new data is written to other locations. To make room for new write operations, GC is necessary to reclaim these invalid physical pages. However, GC operations significantly affect the performance, since the basic unit of an erase operation is the block [8], [10], [34], [40], [41]. Compressional SSDs have brought new issues to garbage collection. When compressional SSDs issue an update request, it may only partially affect pages in one physical page, making

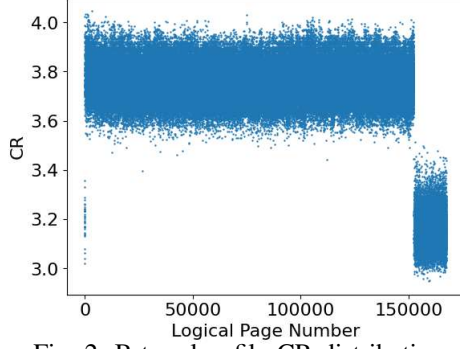


Fig. 2: B-tree log file CR distribution.

this physical page partially valid. Unfortunately, a normal GC operation only considers whether a physical page is valid or not, and it cannot free the invalid space inside these partially valid physical pages. As a result, the invalid space within these partially valid physical pages remains unused, impacting overall storage efficiency.

D. Motivation

Within a specific range, logical pages exhibit similar CRs. Our experiment demonstrates a spatial locality of CRs across pages. Illustrated in Figure 2, towards the beginning of the Logical Page Number (LPN) sequence, the majority of pages maintain a CR of approximately 4, while those towards the end exhibit a CR closer to 3. This observation indicates that logical pages within a defined range tend to possess comparable CRs, whereas variations exist across different sections of the page. These findings show the feasibility of consolidating several consecutive logical pages for N-to-1 mapping within a specific range. However, it also underscores the necessity of employing distinct N-to-1 mapping translations, even within a single file.

III. OVERALL ARCHITECTURE

Figure 3 shows the overview of our mapping table design. It mainly contains two parts: the N-to-1 translation pages and the corresponding Global Translation Directory.

A translation page in our design has a similar structure to traditional translation pages, but each entry holds different flags. Each entry stores a physical page address representing N consecutive logical pages. As shown in Figure 3, the translation page stored in PPN 302 is a 2-to-1 translation page. It stores mapping entries with an CR of 2, indicating that two consecutive logical pages are compressed into one physical page. For instance, logical pages 0 and 1 are compressed and stored in physical page 405. Therefore, the first entry in this translation page stores PPN 405. To accommodate different CRs, we maintain four types of N-to-1 translation pages within the system. These types are defined by the value of N, which can be set as 1, 2, 4, 8 in this paper.

The Global Translation Directory is similar to the traditional mapping table, but with two additional flags: CR (Compression Ratio) and OF (Offset), while the logical space range of each entry remains unchanged. CR represents the compression ratio of the corresponding translation page, while OF indicates the offset of the associated Translation Chunk within the translation page. For example, in Figure 3, if the physical page size is

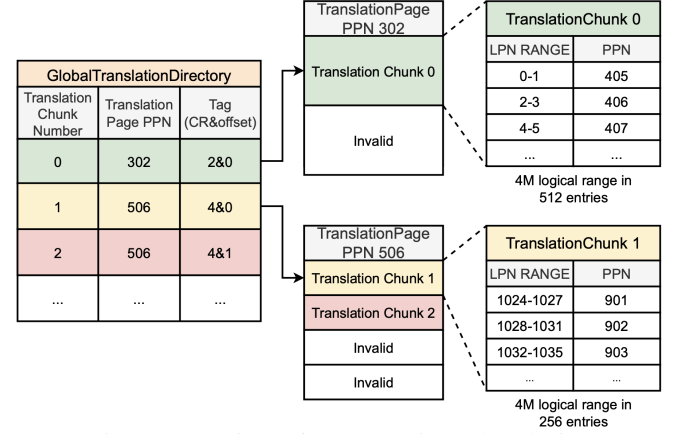


Fig. 3: Overview of our mapping table design.

4KB, each directory entry maps to a 4MB logical address. In traditional mapping tables, one directory entry corresponds to one translation page. However, with N-to-1 translation pages, each page encapsulates N*4MB of logical space. Therefore, the page can be divided into N distinct Translation Chunks, each representing 4MB of logical space. Each directory entry aligns with a Translation Chunk, facilitating efficient organization and mapping of logical addresses within the flash memory system. The Translation Chunk 0 occupies the first chunk of the Translation Page 302 with compression ratio 2, so its offset is 0 and CR is 2 in Tag. The CR and OF tags help the FTL efficiently locate the position of the associated Translation Chunk. The FTL can swiftly determine the physical page number corresponding to any given logical page number. This process ensures rapid and accurate mapping between logical and physical addresses.

IV. DETAILED TECHNIQUES

In this section, we present our memory-efficient mapping table design in detail. First, to make full use of the space in N-to-1 translation pages, we propose a partial allocation scheme for managing translation pages to avoid logical space fragments. Next, we introduce demotion and promotion schemes to handle the CR inconsistencies in update requests. Additionally, the reusable allocation scheme reuses the invalid space caused by demotion and promotion. Finally, to free up more space, we develop a compression-aware GC.

A. Partial allocation

N-to-1 translation pages map larger logical spaces in one page, making the traditional full allocation strategy ineffective. Figure 4 illustrates a scenario with two write requests. The first write request targets Translation Chunk 0 with a CR of 2. If a full allocation strategy is employed, a 2-to-1 Translation Page 302 would map Translation Chunks 0 and 1. Then the second request targets Translation Chunk 3 with a CR of 4. In this case, Translation Chunks 3-6 are assigned to the 4-to-1 Translation Page 907. This causes fragmentation, as Translation Chunk 2 cannot utilize N-to-1 mapping. Regardless of its compression ratio, a full translation page is still needed, leading to inefficiency.

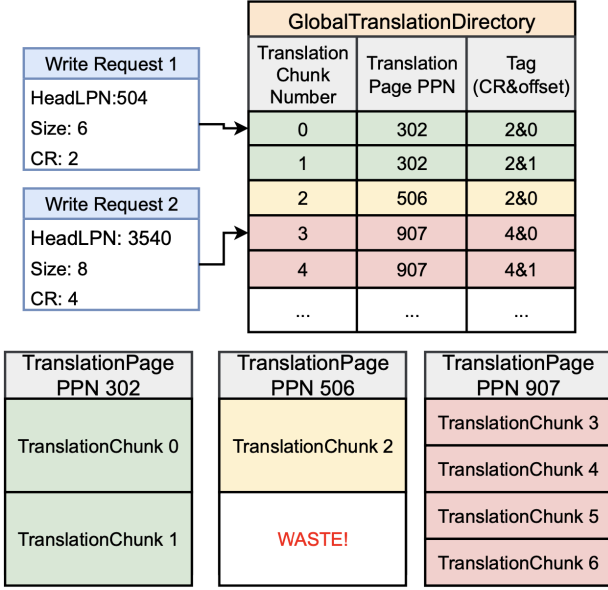


Fig. 4: Illustration of the full allocation issue.

To address this problem, a partial allocation strategy is employed. Partial allocation assigns only part of the translation page for each Translation Chunk. For instance, if a Translation Chunk requires the allocation of translation pages to store mapping entries with a CR of 2, only half of the 2-to-1 translation page is used, and the remaining half is available for subsequent allocations. The CR and OF tags in the directory enable efficient retrieval of the physical page numbers (PPNs) without any performance degradation. This approach ensures the benefits of N-to-1 mapping while minimizing wasted space and improving overall efficiency.

B. Demotion and Promotion

The update write requests may modify the CR of the same logical pages. If the new CR is lower than that of the Translation Chunk, a demotion function is invoked. On the other hand, if the new CR is higher, the system checks if the Translation Chunk can be promoted. If the Translation Chunk meets the requirements for promotion, a promotion scheme is executed.

In the first situation, new PPNs cannot be accommodated within the existing translation page. This occurs because an entry must store multiple PPNs when the new CR is lower than the previous one. For instance, in Figure 5, TranslationChunk 2 is stored in TranslationPage 506 with a CR of 4, where each entry can map 4 logical pages. If an update write request with a new CR 2 is received, LPN 2528-2531 can no longer be compressed into a single physical page. Instead, 2 PPNs are required for these 4 logical pages. In this case, a demotion scheme is invoked. This operation allocates a new chunk of an N-to-1 translation page for the lower CR (e.g., TranslationPage 302 with a CR of 2). The entry from the old translation page is copied to the new translation page. As shown in Figure 5, the updated mappings in LPN 2528-2531 are stored in two entries in new translation page, while the unchanged mapping for LPN 2532-2535 is directly copied to the new place.

In the second situation, if the new CR of the update write request is higher than the translation page CR, we check

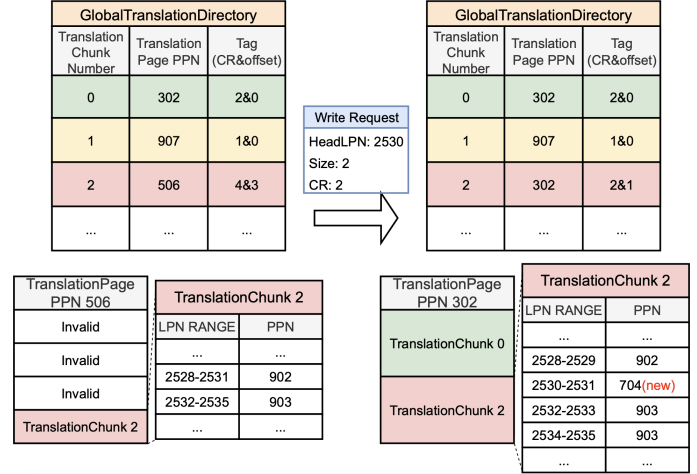


Fig. 5: Demotion with partial allocation.

whether every two entries in the Translation Chunk store the same PPN. If so, the chunk can be promoted to a higher CR. This condition may appear strict but is often met in practice. Demotion typically modifies only a small number of PPNs, leaving most unchanged. During demotion, most PPNs are copied to the new place, just like LPN 2532-2535 in Figure 5. If the modified entries LPN 2528-2531 are updated and store the same PPN with a higher CR again, the whole chunk is promotable. In this scenario, we allocate a new chunk of an N-to-1 translation page for the higher CR and move the entries from the old chunk. This approach reduces memory usage.

- **Reusable Allocation.** Partial allocation enables new translation pages to be allocated at a chunk granularity. However, during demotion and promotion schemes, moving entries to new chunks invalidates the old chunks, leading to an accumulation of invalid chunks in older translation pages. Over time, this accumulation reduces the overall memory efficiency of the system. In the worst case, our memory-efficient table needs the same number of translation pages as the traditional mapping table even if we use partial allocation. Reusing these invalid chunks helps reduce memory usage and minimizes the frequency of write-back operations. Therefore, we propose a reusable allocation scheme. We collect all the invalid chunks of different types of N-to-1 translation pages and reuse them when a new allocation request comes in. In detail, a free space list is maintained for each type of N-to-1 translation page to track free space. Invalid chunks from translation pages are added to the corresponding list during promotion and demotion. For a new allocation request, the corresponding list is checked for a reusable chunk. If the list is empty, the FTL allocates a new N-to-1 translation page and adds its chunks to the list. Furthermore, when the cache is full and the eviction is needed. We will first check if there is enough free space in memory to store all the valid and dirty parts in the evicted pages. If yes, we just allocate these valid and dirty parts to those free spaces in memory instead of writing back to NAND flash. If not, we will trigger a normal writeback operation.

C. Compression-aware GC

Partially valid data may use a whole physical page, which severely degrades the GC performance. As we can see in

Figure 5, when a new update write request triggers a demotion operation, the data in PPN 902 becomes partially valid because the newly written data can not be compressed together. Normal GC can not release these invalid spaces by reallocating these partially valid pages to other blocks. In the worst case, one physical page may hold only one logical page with its CR is 8. As time goes on, less physical space can be released in one GC operation.

To address this issue, we propose a compression-aware GC. For each data block, we track valid physical page numbers and the valid logical page numbers for GC. Given a block to erase, we check these two numbers in this block. If the valid physical page number is low, normal GC performs well, and we execute it as well. However, if the number of valid physical pages is high and the number of valid logical pages is low, normal GC operation performs poorly, and compression-aware GC is triggered. Reallocation is delayed and the data is kept in memory. Then we directly erase the victim block to release the space immediately. The data waits for a new write request to attempt recompression of consecutive logical pages into one page. If recompression succeeds, we also check if the translation page chunk can be promoted.

V. EVALUATION

We evaluate our mapping schemes on a 512GB SSD using MQSim, a state-of-the-art open-source SSD simulator. MQSim models all SSD components, including the host interface, SSD controllers, flash controllers, and flash chips. It supports multi-queue SSDs and measures end-to-end latency, making it ideal for our study. To adapt MQSim for our evaluation, we incorporated compression components and assessed overall performance in terms of end-to-end latency. In this section, we first present the setup and workloads and show the experimental results in terms of mapping table size, the cache hit ratio, and the latency. We compare our design with optimal-FTL, DFTL-base and DFTL-prefetch. In the optimal-FTL scenario, we assume the entire mapping table fits within DRAM. For DFTL-prefetch, a cache miss triggers the prefetching of the entire translation page, mitigating subsequent translation reads within the same page and effectively reducing translation page reads from cache misses. Subsequently, we evaluate our compression-aware GC scheme, and explore the performance impact with different cache sizes and large SSDs. At last, we analyze overheads of our design.

A. Experiment Setup

The parameters of the 512GB SSD are shown in Table I. In the evaluation, the mapping cache is set as large as the mapping table of a block-level FTL, which is 2MB. Other cache sizes are also considered. Furthermore, in the evaluation, we use real-world workloads to study the performance of DFTL and our design. These traces are collected from filebench [25]. The workload characteristics can be found in Table II. However, as these traces lack actual data, we utilize the SDGen scheme [9] to sample data from real applications, leveraging this information to generate CR information for the traces. In

the next section, we show the performance of these FTL designs with different workloads.

TABLE I: The SSD parameters.

Parameter	Value	Parameter	Value
Channel	8	Page size	4KB
Chip	4	read latency	75us
Die	2	write latency	750us
Plane	2	erase latency	3.8ms
Block	2048	compression latency	50us
Page	256	decompression latency	2.5us

TABLE II: Workloads information.

Workloads	Number of Read Request	Average Read Length (4KB pages)	Number of Write Request	Average Write Length (4KB pages)
fileserver_ext4	141542	105.18	545689	37.83
fileserver_f2fs	346203	45.57	274453	90.78
varmail_ext4	121202	17.74	355954	10.25
varmail_f2fs	177277	17.22	253410	23.27

B. Overall Performance

1) *Mapping Table Size*: Our design significantly reduces the mapping table size compared with the DFTL, which only caches the translation page without reducing the mapping table size. By using DFTL as the baseline, we normalize other methods' table sizes for comparison. As shown in Figure 6(a), DFTL's 1-to-1 mapping corresponds to a value of 1. When implementing N-to-1 mapping with full allocation, only a 15% reduction in mapping table size is achieved. This approach leads to significant wastage of translation page space due to inefficient allocation strategies. In contrast, N-to-1 mapping with partial and reusable allocation strategies can significantly reduce the mapping table size. On average, the size of the whole N-to-1 mapping table is reduced by 50%.

2) *Cache Hit Ratio*: Our approach also leads to a notable increase in the cache hit ratio. Illustrated in Figure 6(b), our cache hit ratio surpasses that of DFTL by approximately 50%. In a 1-to-1 mapping scenario, a single translation page can only represent 4MB of logical space, whereas an N-to-1 mapping configuration can accommodate $N \times 4\text{MB}$ of logical space. Leveraging this increased capacity, our design can store more mapping information within the cache. Therefore, we achieve a higher cache hit ratio and reduce the flash read and flash write operation numbers for mapping.

3) *Latency*: With the advantages of a reduced-size mapping table, our design exhibits improved latency performance. Figure 6(c) illustrates the end-to-end latency across various workloads. DFTL-base, with the poorest performance, is normalized to 1, and other methods are expressed as a fraction of its latency. The results demonstrate that our N-to-1 mapping approach delivers superior performance compared to both DFTL-base and DFTL-prefetch. Specifically, our N-to-1 mapping scheme achieves 5-10% performance improvements compared to DFTL-prefetch. Moreover, our design showcases a notable 2.06x speedup compared to DFTL-base.

4) *GC Performance*: To evaluate the performance of the compression-aware GC technique, we measure the number of GC operations across various workloads. As shown in Figure

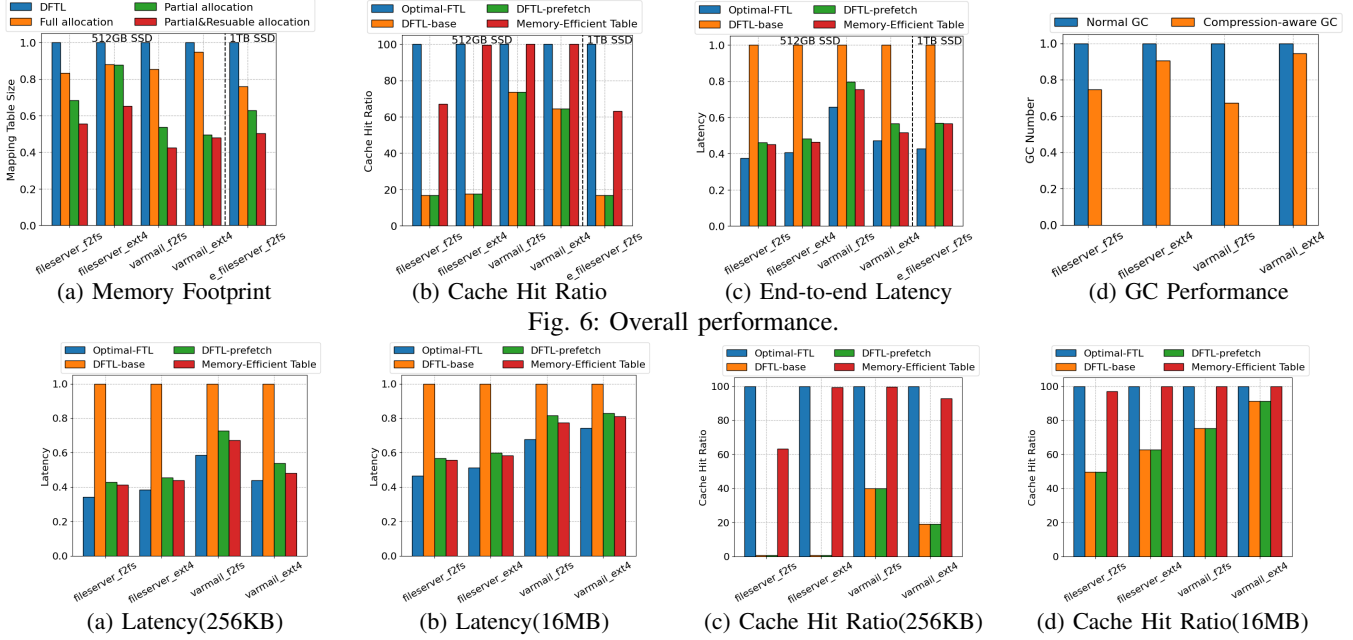


Fig. 6: Overall performance.

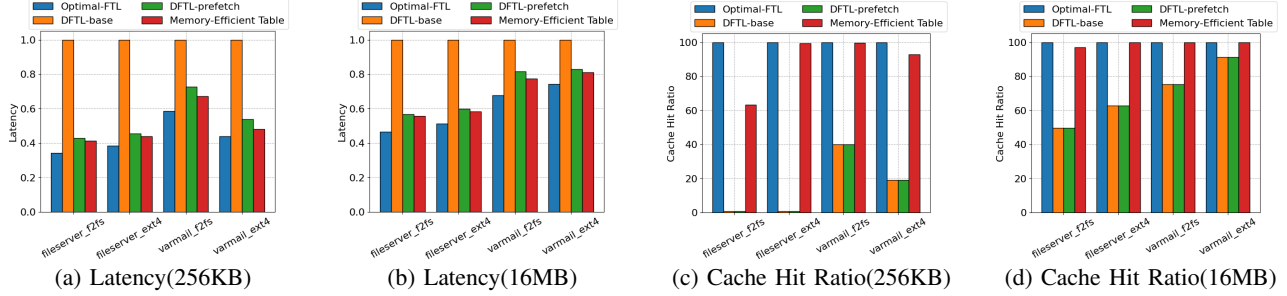


Fig. 7: Performance with different cache sizes.

6(d), we establish the value of normal GC as 1, with the value of our design derived by dividing our GC numbers by the normal GC numbers. Our compression-aware GC approach results in approximately 22% fewer GC operations compared to normal GC. It shows that our compression-aware GC can free more physical space and enhance the performance.

C. Detailed Comparison

1) *Cache size*: This section explores the impact of cache size. We investigate two cache sizes: 256KB and 16MB. At 256KB, DFTL's cache hit ratio is low but stabilizes at 16MB. Figure 7 presents the performance of our design across cache sizes. With very small cache sizes, our design achieves a 2.2x latency speedup over DFTL-base. Our cache hit ratio exceeds DFTL's for small caches due to our memory-efficient N-to-1 mapping. Even with larger caches, our design maintains an average 1.6x speedup. The result shows our N-to-1 mapping scheme can work well even if the cache size is very small.

2) *Large workloads in Large SSD*: As the capacities of SSDs continue to expand, understanding the performance of our design in the context of larger SSDs becomes crucial. With a 1TB SSD environment, we scaled up the fileserver_f2fs workloads by doubling both the logical address and the request sizes. As Figure 6 shows, within a 1TB SSD setup, our cache hit ratio remains significantly superior to that of DFTL. Notably, our design achieves a 1.76x speedup compared to DFTL-base, showcasing the efficiency and scalability of our approach in handling larger SSD capacities.

D. Overhead Analysis

1) *Global Translation Directory size*: As above-mentioned, the logical space size represented by a single GTD entry unchanged. Therefore, when a compressional SSD exposes more logical space to the user, the size of the GTD increases proportionally. It is worth noting that the GTD size is relatively

small in comparison. One GTD entry can represent 4MB of logical space with a 4KB page size. Given a total logical space of 512GB, there are 131072 entries in the Global Translation Directory. 4B are needed for each entry, so the entire GTD would only require 512KB, whereas the mapping table size would amount to 512MB if utilizing a 1-to-1 mapping strategy. It shows that even if the Global Translation Directory size doubles in compressional SSDs, it is still very small.

2) *Tag size*: In our N-to-1 mapping design, given a total logical space of 512GB, one byte is enough to store one Tag. Therefore, our design requires 128KB space to store the Tag information, which is negligible compared with the mapping table.

VI. CONCLUSION

In this paper, we propose a memory-efficient N-to-1 L2P mapping table design by reducing the size of the mapping table and enhancing latency performance. In our design, we have devised the demotion and promotion schemes, along with the partial and reusable allocation methods to fully utilize the N-to-1 translation page. Furthermore, we propose a compression-aware GC strategy to minimize the number of copy operations for partial-invalid physical pages. Our design has been implemented in MQsim, and the experimental results demonstrate significant improvements in the mapping table cache hit ratio and a reduction in I/O latency compared to traditional compressional SSDs.

ACKNOWLEDGMENT

The work described in this paper is partially supported by the grants from the Research Grants Council of the Hong Kong Special Administrative Region, China (GRF 14202123, GRF 14200224), and the Innovation Capability Support Program of Shaanxi, China (No. 2023-CX-TD-08).

REFERENCES

- [1] MySQL Compression. 2024. <https://dev.mysql.com/doc/refman/8.4/en/innodb-compression-internals.html>.
- [2] Scaleflux. <https://scaleflux.com/>.
- [3] solidigmPLC. <https://news.solidigm.com/en-WW/217006-solidigm-demonstrates-world-s-first-penta-level-cell-ssd-at-flash-memory-summit>.
- [4] M. Bartfk, S. Ubik, and P. Kubalik. LZ4 compression algorithm on FPGA. In *2015 IEEE International Conference on Electronics, Circuits, and Systems (ICECS)*, pages 179–182. IEEE, 2015.
- [5] P. Boncz, T. Neumann, and V. Leis. FSST: fast random access string compression. *Proceedings of the VLDB Endowment*, 13(12):2649–2661, 2020.
- [6] C.-H. Chen, C.-T. Chen, and W.-T. Huang. The real-time compression layer for flash memory in mobile multimedia devices. *Mobile Networks and Applications*, 13:547–554, 2008.
- [7] F. Chen, D. A. Koufaty, and X. Zhang. Understanding intrinsic characteristics and system implications of flash memory based solid state drives. *ACM SIGMETRICS Performance Evaluation Review*, 37(1):181–192, 2009.
- [8] N. Dayan, P. Bonnet, and S. Idreos. GeckoFTL: Scalable flash translation techniques for very large flash devices. In *Proceedings of the 2016 International Conference on Management of Data*, pages 327–342, 2016.
- [9] R. Gracia-Tinedo, D. Harnik, D. Naor, D. Sotnikov, S. Toledo, and A. Zuck. SDGen: Mimicking datasets for content generation in storage benchmarks. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 317–330, 2015.
- [10] J. Guo, Y. Hu, B. Mao, and S. Wu. Parallelism and garbage collection aware I/O scheduler with improved SSD performance. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1184–1193. IEEE, 2017.
- [11] A. Gupta, Y. Kim, and B. Ugaonkar. DFTL: a flash translation layer employing demand-based selective caching of page-level address mappings. *Acm Sigplan Notices*, 44(3):229–240, 2009.
- [12] J. Hao, X. Chen, Y. Qiao, Y. Zhang, and T. Zhang. Implementing flash-cached storage systems using computational storage drive with built-in transparent compression. In *2021 IEEE International Conference on Networking, Architecture and Storage (NAS)*, pages 1–8. IEEE, 2021.
- [13] X. Hu, F. Wang, W. Li, J. Li, and H. Guan. {QZFS};{QAT} accelerated compression in file system for application agnostic and cost efficient data storage. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 163–176, 2019.
- [14] K. Huang, Z. Shen, Z. Shao, T. Zhang, and F. Chen. Breathing new life into an old tree: Resolving logging dilemma of B+-tree on modern computational storage drives. *Proceedings of the VLDB Endowment*, 17(2):134–147, 2023.
- [15] C. Ji, L.-P. Chang, L. Shi, C. Gao, C. Wu, Y. Wang, and C. J. Xue. Lightweight data compression for mobile flash storage. *ACM Transactions on Embedded Computing Systems (TECS)*, 16(5s):1–18, 2017.
- [16] S. Jiang, L. Zhang, X. Yuan, H. Hu, and Y. Chen. S-FTL: An efficient address translation for flash memory by exploiting spatial locality. In *2011 IEEE 27th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–12. IEEE, 2011.
- [17] M. Kang, W. Lee, J. Kim, and S. Kim. PR-SSD: Maximizing partial read potential by exploiting compression and channel-level parallelism. *IEEE Transactions on Computers*, 72(3):772–785, 2022.
- [18] D. Kim, Y. Won, J. Cha, S. Yoon, J. Choi, and S. Kang. Exploiting compression-induced internal fragmentation for power-off recovery in SSD. *IEEE Transactions on Computers*, 65(6):1720–1733, 2015.
- [19] S. Lee, J. Park, K. Fleming, J. Kim, et al. Improving performance and lifetime of solid-state drives using hardware-accelerated compression. *IEEE Transactions on consumer electronics*, 57(4):1732–1739, 2011.
- [20] Q. Li, L. Shi, R. Pan, C. Ji, X. Li, and C. J. Xue. Selective compression scheme for read performance improvement on flash devices. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*, pages 43–50. IEEE, 2018.
- [21] Q. Li, Q. Xiang, Y. Wang, H. Song, R. Wen, W. Yao, Y. Dong, S. Zhao, S. Huang, Z. Zhu, et al. More than capacity: Performance-oriented evolution of pangu in alibaba. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 331–346, 2023.
- [22] W. Liu, F. Mei, C. Wang, M. O'Neill, and E. E. Swartzlander. Data compression device based on modified LZ4 algorithm. *IEEE Transactions on Consumer Electronics*, 64(1):110–117, 2018.
- [23] D. Ma, J. Feng, and G. Li. LazyFTL: A page-level flash translation layer optimized for NAND flash memory. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, pages 1–12, 2011.
- [24] B. Mao, S. Wu, H. Jiang, Y. Yang, and Z. Xi. EDC: Improving the performance and space efficiency of flash-based storage systems with elastic data compression. *IEEE Transactions on Parallel and Distributed Systems*, 29(6):1261–1274, 2018.
- [25] R. McDougall and J. Mauro. Filebench. URL: <http://www.nfsv4bat.org/Documents/nasconf/2004/filebench.pdf> (Cited on page 56.), 2005.
- [26] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis. Dremel: interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment*, 3(1-2):330–339, 2010.
- [27] Y. Pan, H. Chen, J. Zhao, and Y. Xu. HCFTL: A locality-aware flash translation layer for efficient address translation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(8):2477–2489, 2021.
- [28] Y. Pan, Y. Li, H. Zhang, and Y. Xu. Lifetime-aware FTL to improve the lifetime and performance of solid-state drives. *Future Generation Computer Systems*, 93:58–67, 2019.
- [29] Y. Pan, M. Lin, Z. Wu, H. Zhang, and Z. Xu. Caching-aware garbage collection to improve performance and lifetime for NAND flash SSDs. *IEEE Transactions on Consumer Electronics*, 67(2):141–148, 2021.
- [30] Y. Park and J.-S. Kim. zFTL: Power-efficient data compression support for NAND flash-based consumer electronics devices. *IEEE transactions on consumer electronics*, 57(3):1148–1156, 2011.
- [31] Y. Qiao, X. Chen, N. Zheng, J. Li, Y. Liu, and T. Zhang. Closing the B+-tree vs. LSM-tree write amplification gap on modern storage hardware with built-in transparent compression. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 69–82, 2022.
- [32] Y. Song, Y. Huang, Y. Lv, Y. Zhang, and L. Shi. When F2FS meets compression-based SSD! In *Proceedings of the 15th ACM Workshop on Hot Topics in Storage and File Systems*, pages 87–92, 2023.
- [33] A. Tavakkol, J. Gómez-Luna, M. Sadrosadati, S. Ghose, and O. Mutlu. MQSim: A framework for enabling realistic studies of modern Multi-Queue SSD devices. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pages 49–66, 2018.
- [34] S. Wu, Y. Lin, B. Mao, and H. Jiang. GCaR: Garbage collection aware cache management with improved performance for flash-based SSDs. In *Proceedings of the 2016 International Conference on Supercomputing*, pages 1–12, 2016.
- [35] Z. Yang, Q. Xu, S. Gao, C. Yang, G. Wang, Y. Zhao, F. Kong, H. Liu, W. Wang, and J. Xiao. OceanBase Paetica: A Hybrid Shared-Nothing/Shared-Everything Database for Supporting Single Machine and Distributed Cluster. *Proceedings of the VLDB Endowment*, 16(12):3728–3740, 2023.
- [36] X. Yao, Q. Li, K. Lin, X. Gan, J. Zhang, C. Gao, Z. Shen, Q. Xu, C. Yang, and J. Xue. Extremely-Compressed SSDs with I/O Behavior Prediction. *ACM Transactions on Storage*, 20(4):1–38, 2024.
- [37] K. S. Yim, H. Bahn, and K. Koh. A flash compression layer for smartmedia card systems. *IEEE Transactions on consumer Electronics*, 50(1):192–197, 2004.
- [38] F. Zhang, W. Wan, C. Zhang, J. Zhai, Y. Chai, H. Li, and X. Du. CompressDB: Enabling efficient compressed data direct processing for various databases. In *Proceedings of the 2022 International Conference on Management of Data*, pages 1655–1669, 2022.
- [39] X. Zhang, J. Li, H. Wang, D. Xiong, J. Qu, H. Shin, J. P. Kim, and T. Zhang. Realizing transparent os/apps compression in mobile devices at zero latency overhead. *IEEE Transactions on Computers*, 66(7):1188–1199, 2017.
- [40] B. Zhou, S. Wan, and C. Xie. Isolation: Inexpensively separating cold data via garbage collection to improve the lifetime and performance of nand flash ssds. *Concurrency and Computation: Practice and Experience*, 33(15):e5460, 2021.
- [41] K. Zhou, S. Hu, P. Huang, and Y. Zhao. Lx-ssd: Enhancing the lifespan of nand flash-based memory via recycling invalid pages. In *Proc. 33rd Int. Conf. Massive Storage Syst. Technol.(MSST)*, pages 1–13, 2017.
- [42] Y. Zhou, F. Wu, P. Huang, X. He, C. Xie, and J. Zhou. An efficient page-level FTL to optimize address translation in flash memory. In *Proceedings of the Tenth European Conference on Computer Systems*, pages 1–16, 2015.
- [43] A. Zuck, S. Toledo, D. Sotnikov, and D. Harnik. Compression and SSDs: Where and How? In *2nd Workshop on Interactions of NVM/Flash with Operating Systems and Workloads (INFLOW 14)*, 2014.